



4 – Applicazioni client-side con REACT

Architetture Web

elementi principali

Third Party
Services

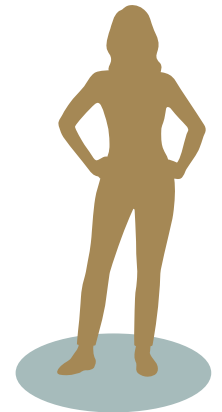
Persistence



Application
Business Logic

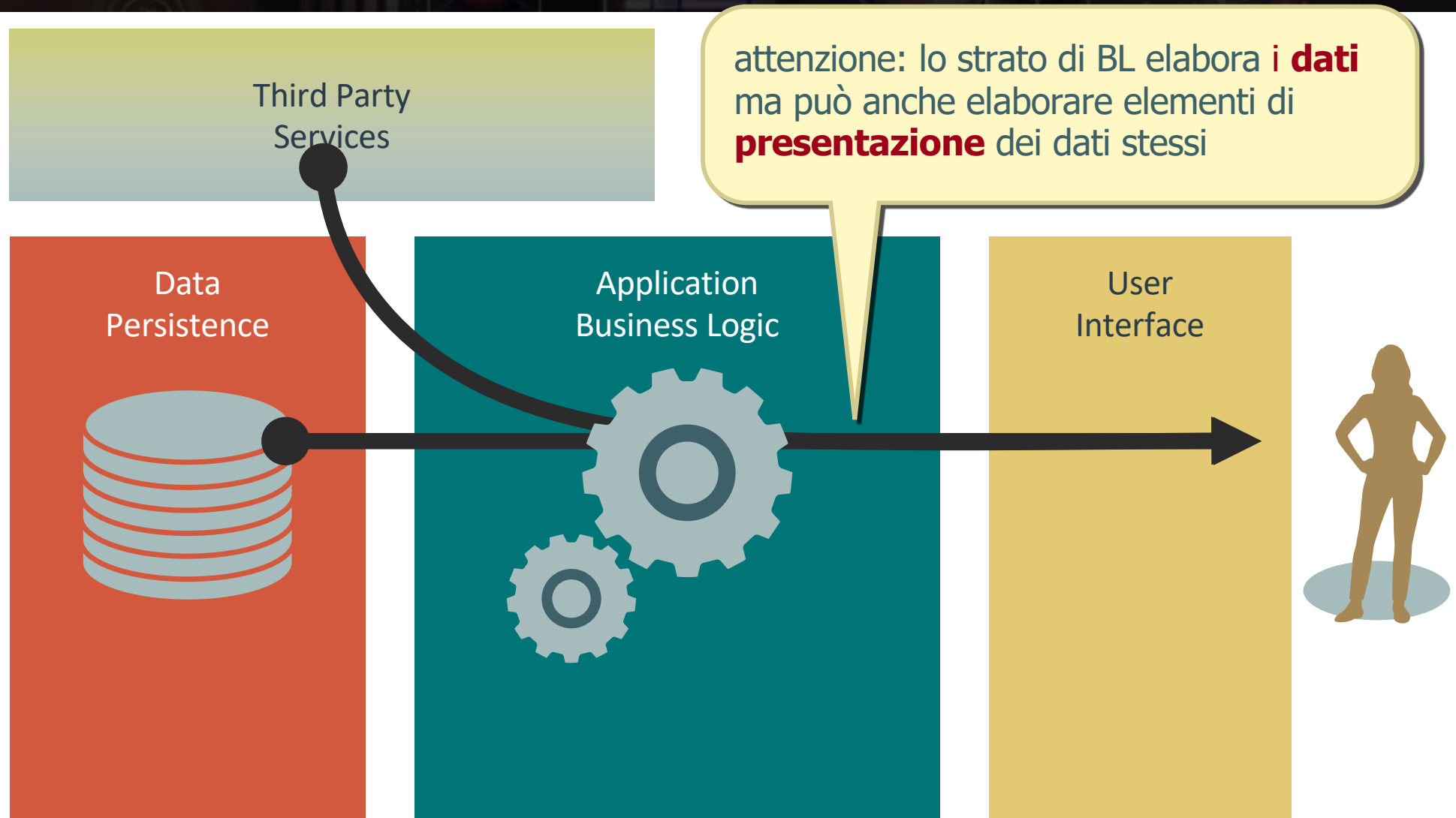


User
Interface



Architetture Web

flow 1: retrieval + presentation



Architetture Web

flow 2a: user action, UI-ONLY

Third Party
Services

per esempio: sposta un elemento sullo
schermo, seleziona qualcosa, ecc.

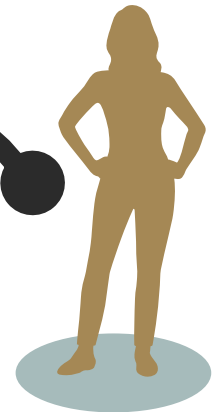
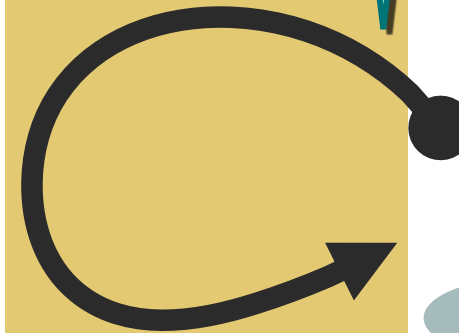
Data
Persistence



Application
Business Logic

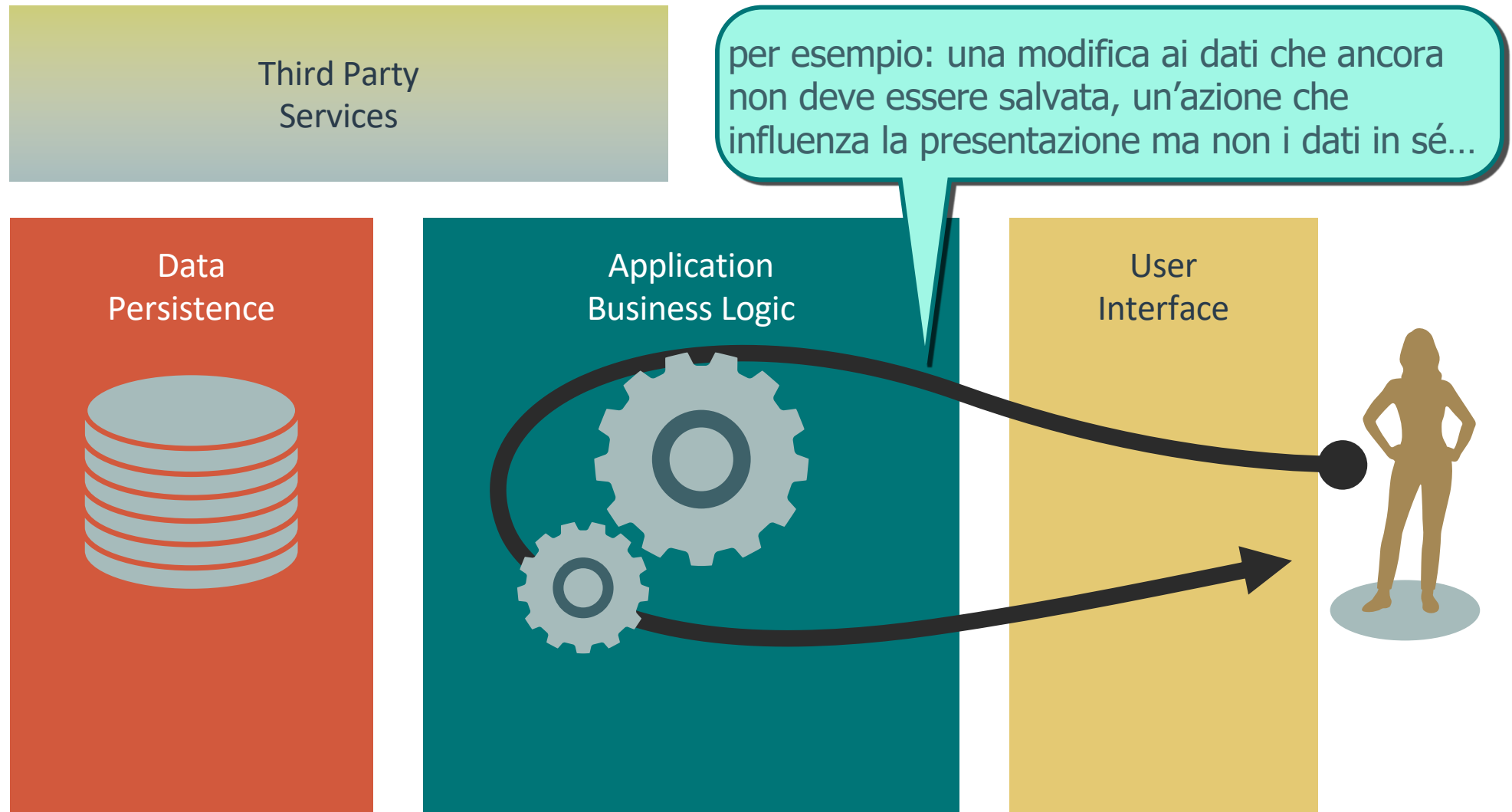


User
Interface



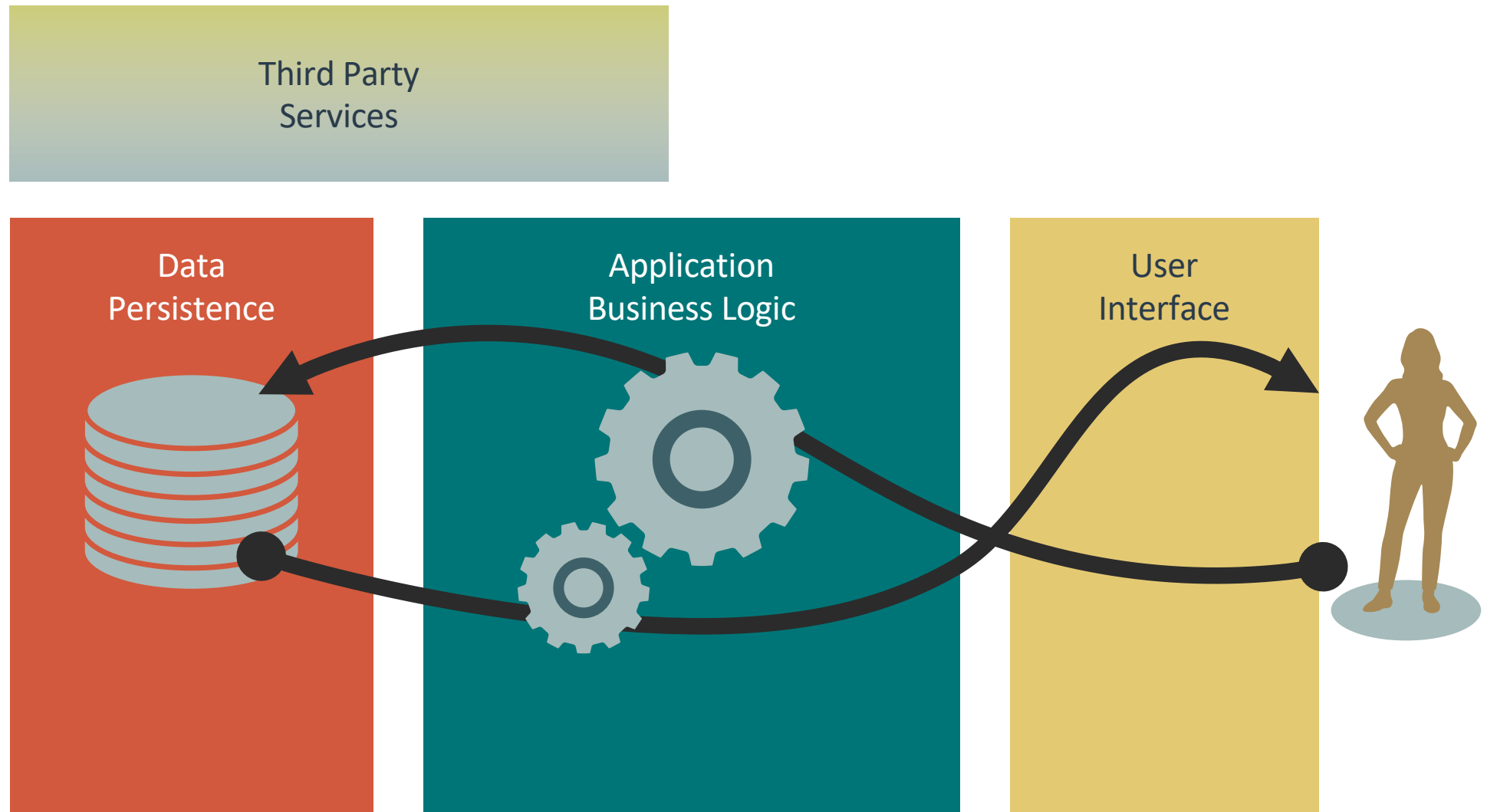
Architetture Web

flow 2B: user action, non-persistent



Architettura Web

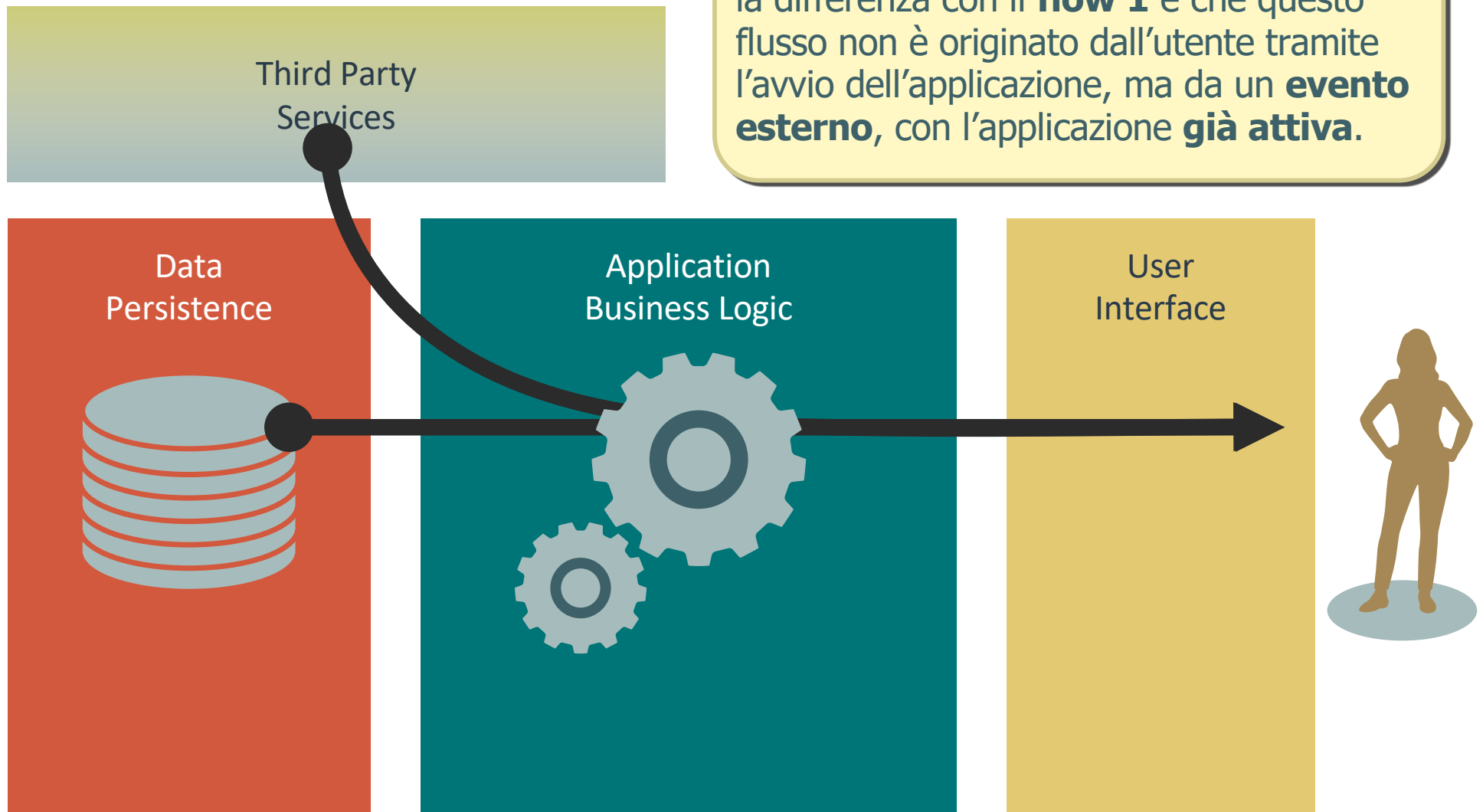
flow 2C: user action with persistent change



Architetture Web

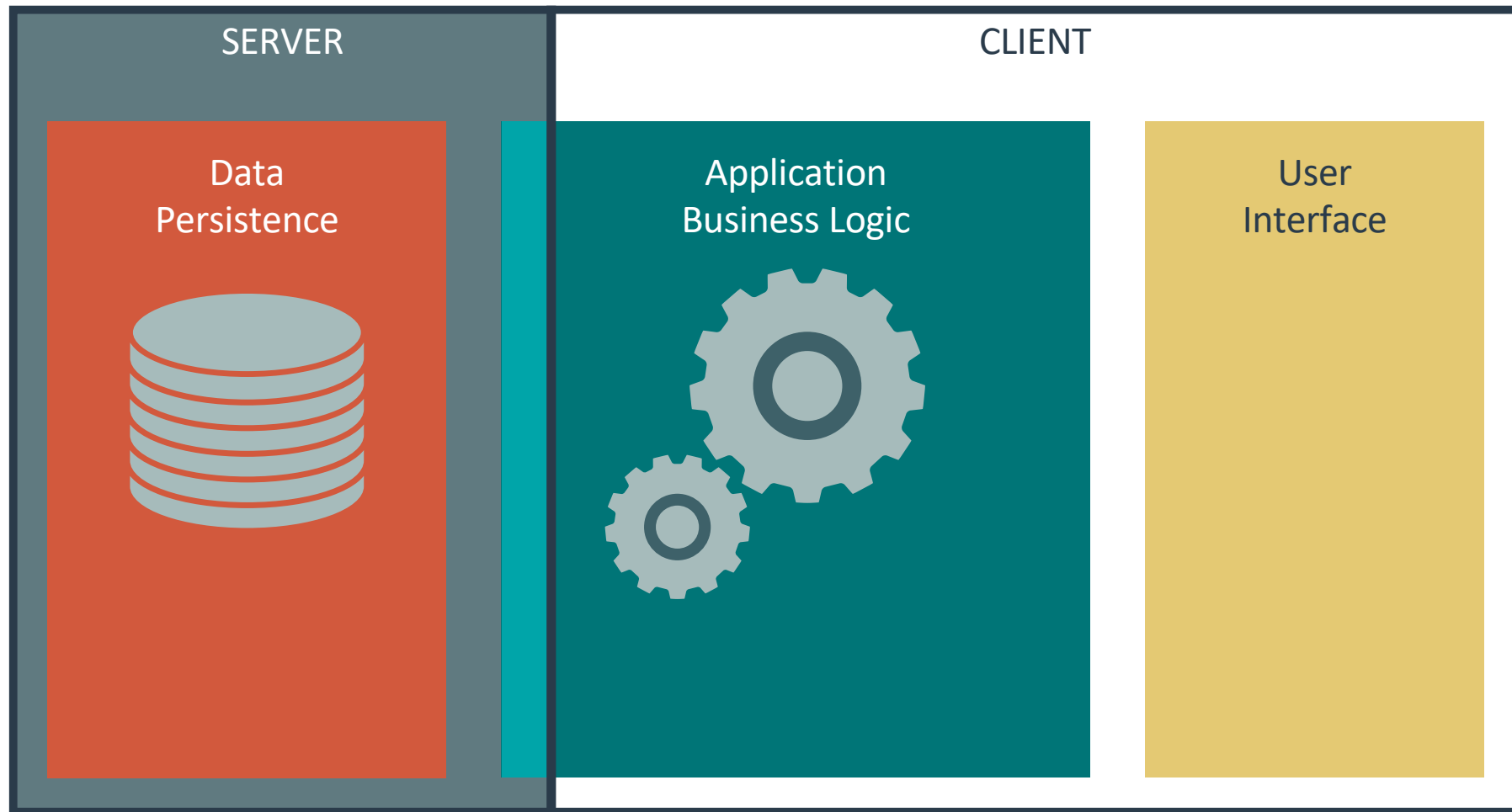
flow 3: external update

la differenza con il **flow 1** è che questo flusso non è originato dall'utente tramite l'avvio dell'applicazione, ma da un **evento esterno**, con l'applicazione **già attiva**.



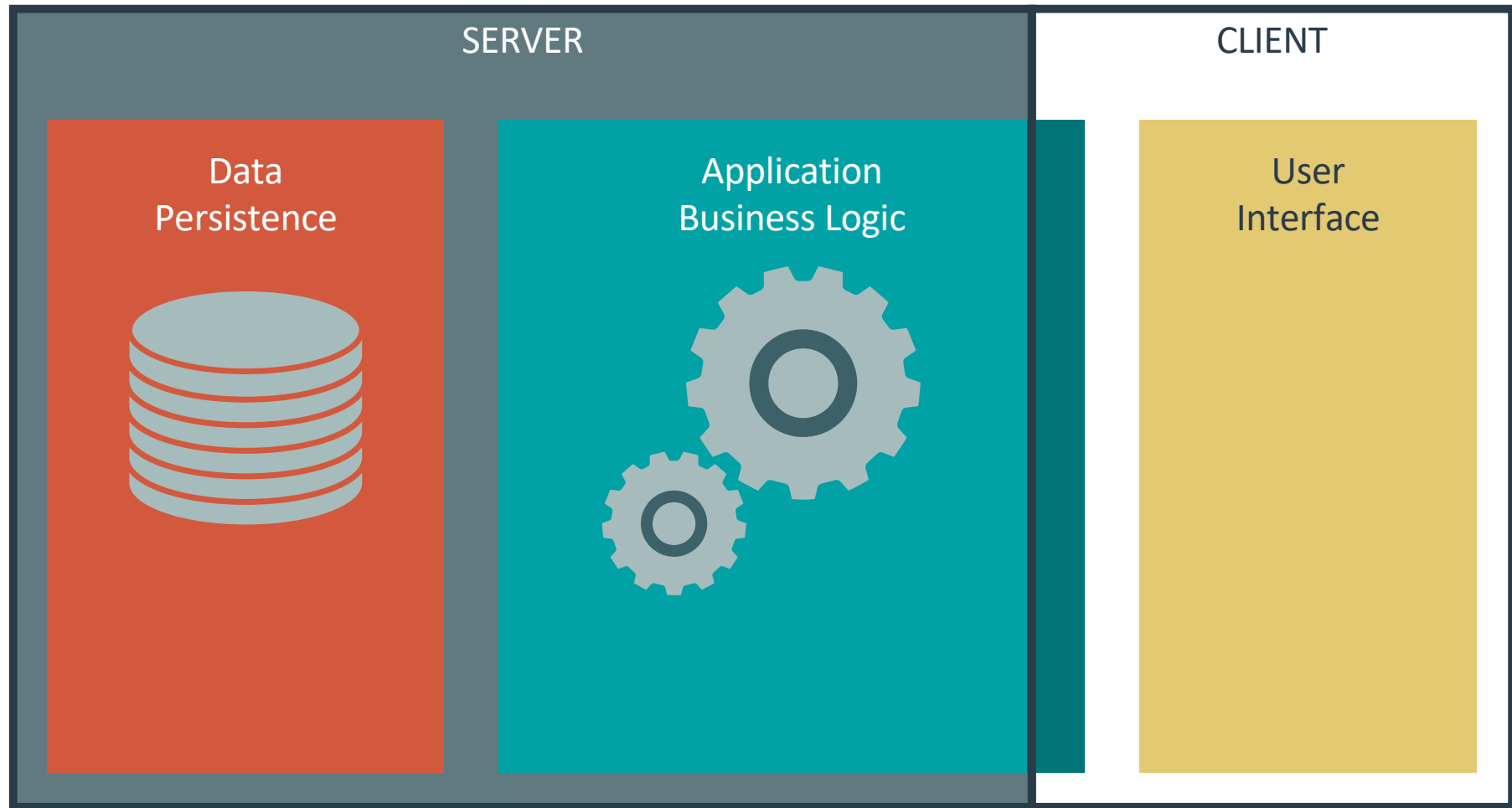
server-side vs client-side

Client-side logic



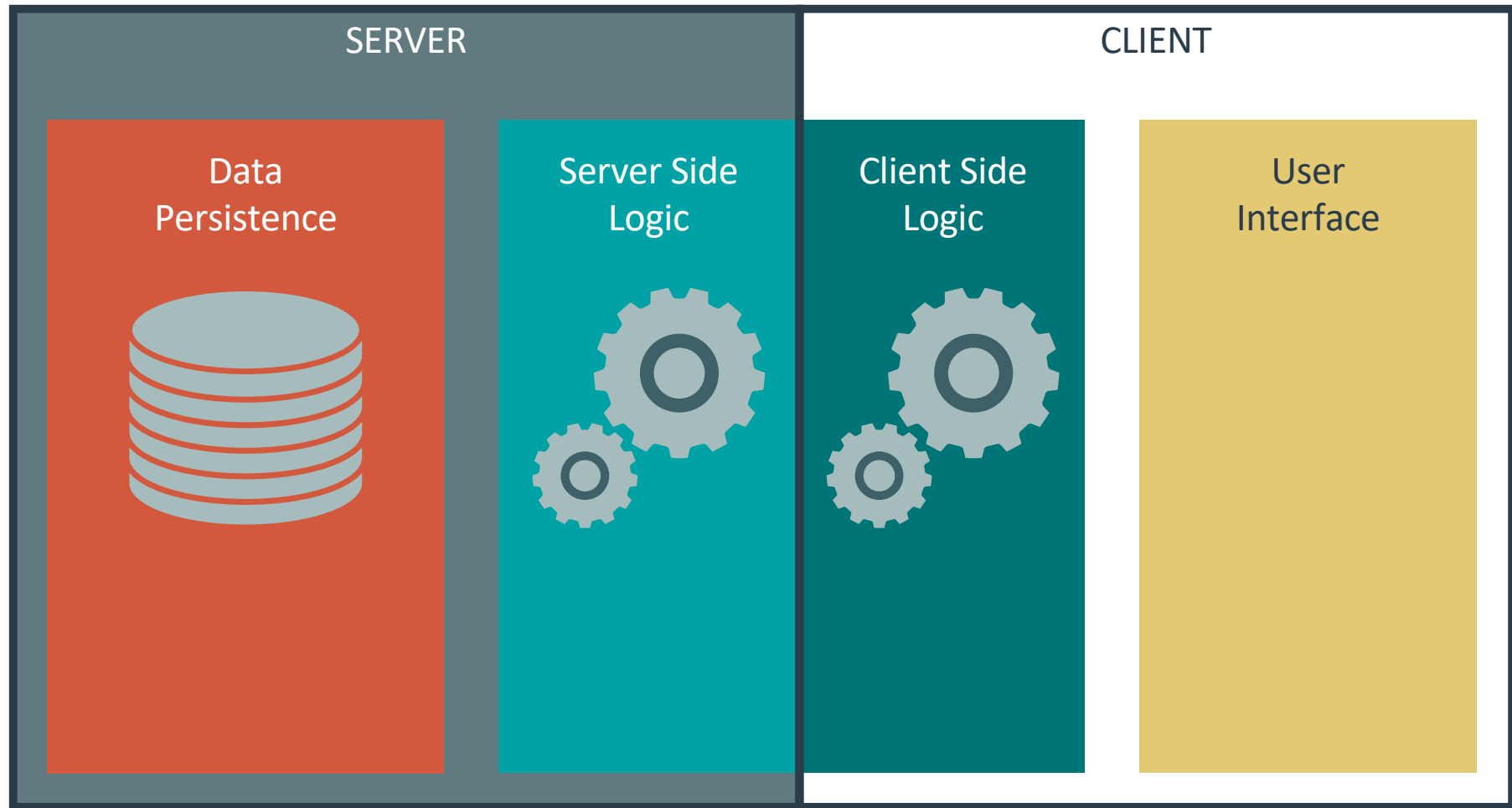
server-side vs client-side

Server-side logic



server-side vs client-side

Two-sided logic



React

Caratteristiche principali

React è propriamente una **libreria**, può essere usato da solo o come parte di un framework che lo supporta («il» framework di React è **Next.JS**)

Noi useremo React come **libreria** in abbinamento a un tool, **Vite**, che in abbinamento a **npm** permette facilmente la configurazione e la gestione di un progetto React

React supporta lo sviluppo di front-end con UI orientate ai **componenti**

In questo, è simile ad altri suoi competitor, come Angular o Vue, che sono tuttavia **framework** a tutto tondo.

React supporta lo sviluppo di **Single Page Application**

Applicazioni client-side costituite da un'unica pagina HTML il cui DOM viene costantemente aggiornato dall'applicazione stessa (tramite le librerie di React).

Framework vs Librerie

Cosa intendiamo quando diciamo che React è di per sé una libreria e non un framework?

Nell'ambito dello sviluppo di front-end, una **libreria** non è molto diversa da quello che abbiamo già utilizzato: un insieme di **package** che è possibile importare nel proprio progetto, utilizzandone le funzionalità. React è una libreria complessa, e come vedremo include un'**estensione** del linguaggio che richiede di **compilare** gli script per tradurli a JS, ma questo accade anche usando semplicemente Typescript.

React include anche un **runtime**, ossia un insieme di oggetti e funzioni che vengono creati all'avvio dell'applicazione e che gestiscono alcuni aspetti del funzionamento dei componenti che noi andremo a creare. Si tratta però sempre di funzionalità che **utilizziamo** dall'interno del nostro codice, che ci facilitano il lavoro, ma non alterano radicalmente i meccanismi di funzionamento di JS o TS.

Un vero e proprio **framework** (quale può diventare React in combinazione con Next.js, ad esempio) di fatto va a realizzare qualcosa di molto simile a una «virtual machine» che va in esecuzione all'avvio dell'applicazione: un **ambiente di esecuzione** «dentro» all'ambiente di esecuzione, che si occupa di caricare ed eseguire il nostro codice, e lo fa in un contesto altamente «modificato». Quando programmiamo dunque dobbiamo essere almeno parzialmente consapevoli dei meccanismi dell'ambiente di esecuzione specifico del framework, che compie svariate operazioni «dietro le quinte».

Si pensi ad esempio al fatto che Angular monitora tutti i cambiamenti nei valori delle proprietà degli oggetti che visualizziamo, mentre Vue ne crea delle copie «addizionate» di metodi e proprietà, per usarle al posto degli oggetti originali.

I framework d'altra parte sono più completi, e supportano lo sviluppo di altri aspetti di un'applicazione client-side oltre alla mera UI. In React spesso questi aspetti sono demandati a librerie separate.

Una differenza importante di React rispetto ad altri framework è che React **non è object-oriented**, ma è fortemente basato sugli **aspetti funzionali** di JS/TS.

Componenti UI

Un componente UI è caratterizzato da:

- un aspetto grafico detto anche **template** del componente: come esso viene visualizzato, che nel caso di un'applicazione web lato client si traduce in una combinazione di HTML e CSS
- una parte o sezione del Data Model che viene visualizzata e/o modificata attraverso il componente stesso.

(per essere precisi, esistono anche componenti che non fanno riferimento al Data Model ma mostrano informazioni statiche, o servono solo a spostarsi da un punto all'altro dell'app, o ancora permettono di mostrare o nascondere informazioni, ossia hanno a che vedere con effetti di interazione che non impattano sul modello dati)

- un comportamento/funzionalità: quali operazioni può compiere l'utente attraverso quel componente, e come si svolge l'interazione?

Se si tratta di un componente «orientato» al Data Model, l'utente potrà visualizzare e/o modificare parti del Data Model attraverso il componente stesso; anche quando non è così tuttavia deve essere possibile esprimere in modo chiaro il «funzionamento inteso» di un componente e la dinamica di interazione con l'utente.

Gerarchie di componenti

Un componente UI può essere «costruito» a partire da altri sottocomponenti: si pensi ad esempio ad un menu di navigazione.

La singola voce del menu è un componente con un comportamento definito: cliccando su di esso, l'utente si sposta in un'altra area dell'applicazione. Tipicamente in un menu compaiono diverse **istanze** di questo componente: stesso comportamento, stesso aspetto, ma contenuto differente.

Il menu intero è costituito da tutte le **istanze** delle voci, e ciascuna di esse mantiene il proprio comportamento, ma il **menu** «costruisce» altro sul comportamento delle varie voci, ad esempio potrà garantire che solo una voce sia selezionata in un dato momento, e tener traccia di quale voce è quella «correntemente» attiva.

Si parla allora di **gerarchie di componenti**.

- Quando un componente è costruito a partire da **istanze di altri componenti** è detto essere **genitore** di quelle istanze
- Le istanze in questione sono dette **figli** del macrocomponente che le contiene.

È importante ricordare che un **componente** in senso astratto non è di per sé **figlio** di un altro componente: sono le sue istanze ad esserlo. Nulla impedisce che istanze diverse di un medesimo componente siano figlie di genitori diversi. La definizione del componente di per sé non dovrà includere informazioni su «chi o cosa» debba contenerlo, ma solo su come chi eventualmente lo conterrà potrà comunicare con esso.

Invece un **componente** in senso astratto può essere detto **genitore**, perché la sua definizione dovrà specificare se e come è costruito, incluse le eventuali istanze di altri sotto-componenti usate al suo interno.

Componenti dinamici

I componenti UI sono dinamici: cambiano il loro aspetto sulla base dei dati e delle interazioni con l'utente. Questo può includere modificare il «set» di sottocomponenti che includono in un dato momento, ossia il «set» di figli.

Ad esempio, un menu che si espande in sotto-voci, potrebbe avere due stati: compresso ed espanso. Nella versione espansa mostra le sotto-voci, che sono altri componenti, dunque suoi figli. Nella versione compressa le sotto-voci scompaiono, dunque dal punto di vista di chi guarda la UI quel componente non ha più alcun figlio.

In generale quando parliamo di **gerarchia di componenti** ci riferiamo alla gerarchia **statica**, ossia quella che riporta tutti i figli che ciascun componente dichiara, indipendentemente da quelli che mostra in un dato momento.

In generale ci sono due modi per far **scompare** e **riapparire** nel componente genitore un componente figlio:

- cambiarne la classe CSS in modo da aggiungere/togliere la proprietà **display:hide**; tecnicamente questo si chiama «mostrare» (show) e «nascondere» (hide) il componente figlio, che resta rappresentato nell'HTML del componente genitore, solo non visibile.
- aggiungerlo/toglierlo dal DOM; tecnicamente questo si chiama «montare» (mount) e «smontare» (unmount) il componente figlio.

Solitamente si preferisce la prima tecnica quando il sotto-componente deve venire acceso/spento di frequente e rapidamente; la seconda quando c'è un sotto-componente sostanzioso che deve essere mostrato solo occasionalmente, o quando la struttura di sotto-componenti viene generata a partire dai dati, e i dati cambiano.

Single Page Application

L'approccio che seguiremo per creare un component-based front-end con React è quello della **Single Page Application**.

Le diverse «schermate» o «pagine» dell'applicazione sono tutte componenti figli di un macro-componente principale, il componente **App**. Il componente **App** monta solo un figlio alla volta. Quando l'utente «naviga» nell'applicazione quello che avviene non è un cambiamento di pagina visualizzata, semplicemente il componente **App** «smonta» un componente-figlio e ne «monta» un altro.

Quando l'utente inizialmente si collega con il browser alla URL corrispondente all'applicazione, il browser scarica la pagina e i relativi script JS, che contengono **l'intera applicazione**. Non c'è mai una navigazione da una pagina ad un'altra. La pagina non conterrà mai dei «veri» collegamenti ipertestuali, che porterebbero il browser ad effettuare un reload.

Ci sono due modi di realizzare questo comportamento nei framework basati su componenti:

- **Gestione manuale**; più semplice da comprendere, può diventare un po' macchinosa su applicazioni molto articolate. Consiste nell'implementare i menu di navigazione come insiemi di «pulsanti» che hanno l'effetto appunto di montare o smontare determinati sotto-componenti del componente principale
- **Gestione tramite router**: si utilizza una libreria (detta appunto **router**) che permette di definire delle route (del tutto analoghe a quelle che abbiamo visto per le server-side app) che verranno gestite dal framework stesso; per ciascuna viene specificato quali componenti devono essere effettivamente montati. Quando l'utente «naviga» ad una di queste route, non si tratta di una vera navigazione che passa per il browser; il **router** intercetta la route e si incarica di mostrare la configurazione di componenti appropriata.

Sebbene usare un **router** sia un po' più complesso e non avremo tempo di affrontare il tema, il vantaggio dell'usarlo (oltre all'automazione di parte dei compiti) è che si preserva all'utente l'illusione di navigare fra pagine diverse; infatti, la barra del browser mostrerà la route attiva anziché la «reale» URL dell'unica «vera» pagina dell'applicazione.

Model-View-ViewModel

Pattern architetturale per UI a componenti

Modello dati:

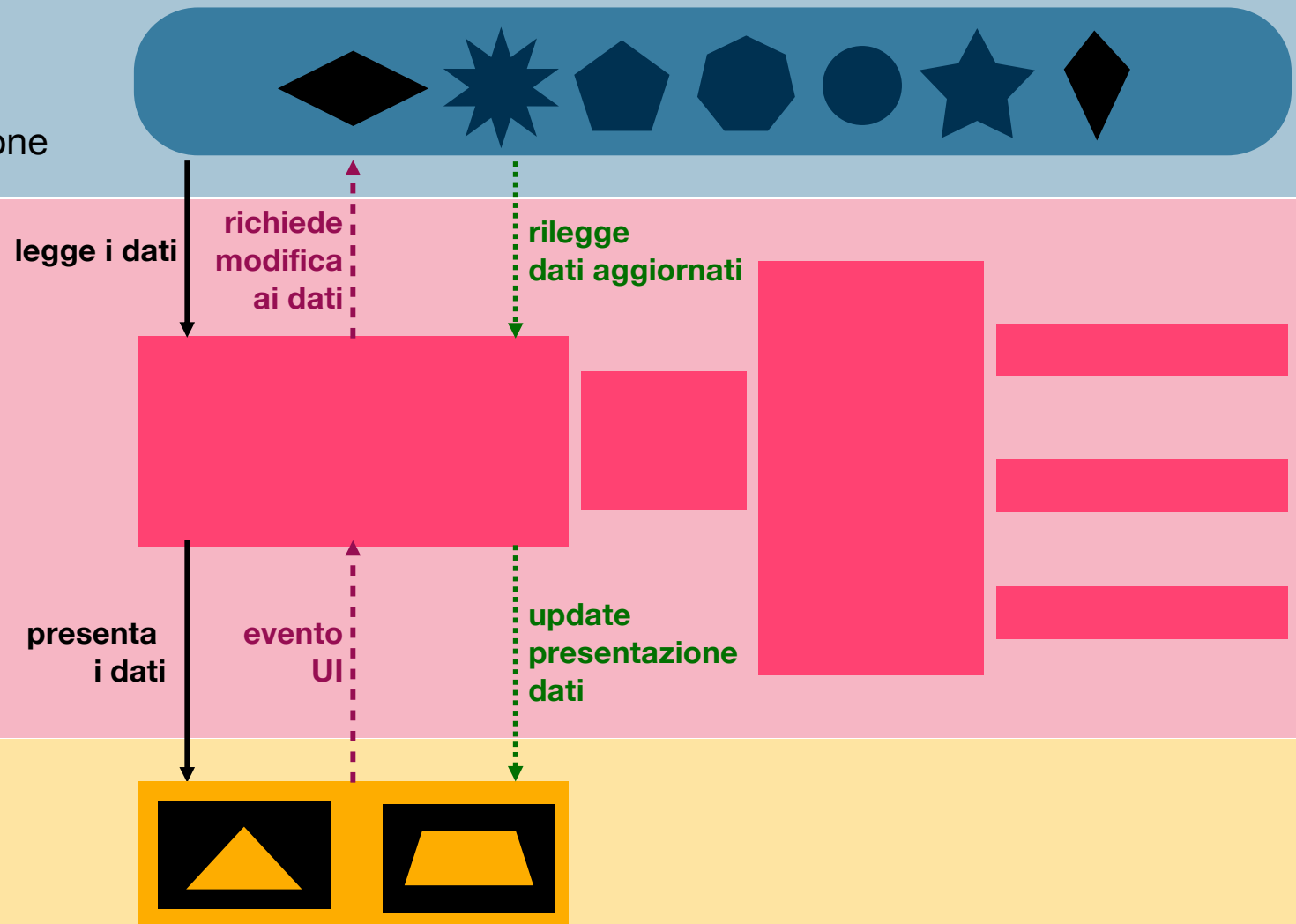
i dati che l'app elabora.
i vincoli che ne regolano
la fruizione e l'elaborazione

View-Model:

il **comportamento**
dei componenti

View:

ciò che l'utente vede
e con cui interagisce
(il DOM)



Property & Event binding

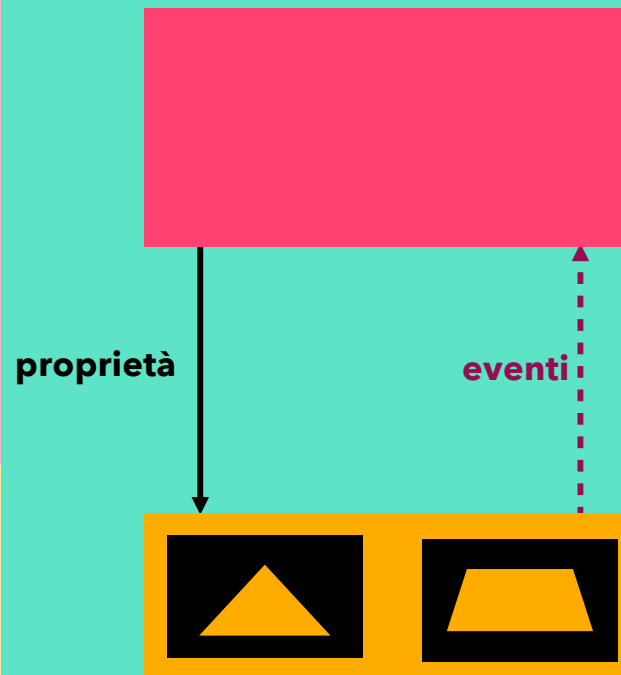
comunicazione fra View-Model e View

Binding (Legame)

Nei framework & affini il collegamento fra view-model e view è parzialmente automatizzato

View-Model:
il **comportamento**
dei componenti

View:
ciò che l'utente vede
e con cui interagisce
(il DOM)



property binding: ogni framework (o affine) adotta la sua strategia per accorgersi di quando qualcosa "cambia" nel view-model e aggiornare di conseguenza la view

event binding: basato sul principio delle callback e degli event listener, permette di associare una funzione a un evento UI

Componenti in React: il ViewModel del MVVM

Un **componente** in **React** è una **funzione** (detta **funzione di rendering**) che **costruisce** il **sottoalbero del DOM** che andrà a costituire la **view** del componente.

- I componenti React sono scritti nel linguaggio JSX/TSX che **estende** Javascript/Typescript, permettendo di interpolare HTML e JS/TS per rendere più efficiente e leggibile la specifica di elementi del DOM. Il linguaggio permette anche di **importare** direttamente i CSS da usare nella definizione del componente.
- La **funzione di rendering** che definisce il componente genera dunque la **view**, e lo fa a partire dai dati che sono rilevanti per quel componente (vedremo fra breve come il componente arriva a «conoscere» tali dati). Il componente deve **dichiarare** (di nuovo, a breve vedremo come) da **quali dati** dipende la sua **view**, e ogni volta che quei dati cambiano React fa in modo che la funzione venga invocata nuovamente, rigenerando la sua view e sostituendo l'elemento corrispondente nel DOM. Questa è la base del **property binding** in React.
- La **funzione di rendering**, nel generare la **view**, specifica anche le **callback** che risponderanno ai vari eventi del DOM, stabilendo pertanto il comportamento del componente. Le **callback** hanno infatti la possibilità di modificare i dati di cui quel componente si occupa (provocando, pertanto, la rigenerazione della view stessa). Questa è la base dell'**event binding** in React.
- Ad ogni componente descritto in JSX/TSX, React associa un **tag** con lo stesso nome del componente. Quel **tag** potrà essere usato nel descrivere la **view** di altri componenti come qualsiasi altro **tag HTML**. In questo modo si va a definire la **gerarchia di componenti** di un'App in React.

Caratteristiche dei componenti React

Come già detto un componente in React è una funzione di rendering, che possiamo **esportare** se vogliamo riferirci a quel componente in un altro sorgente.

Il **nome** che diamo alla funzione diventa il **nome del componente**.

Vedremo in seguito quali parametri **può** prendere in **input** la funzione di rendering, ma può anche non richiederne nessuno. L'**output** della funzione invece **deve** essere un oggetto **ReactElement**, ottenuto tramite un'opportuna espressione JSX/TSX.

Il tipo dell'oggetto-funzione in sé è definito, nelle librerie di React, come **FunctionComponent**.

React richiede che i **componenti** siano **funzioni pure**, ossia:

- Il loro output dipenda solo dal loro input
- Non alterino nel loro codice nessuna struttura dati al di fuori dello scope della funzione stessa

Questo si applica alla funzione di rendering in sé, non alle funzioni di callback utilizzate al suo interno!

ReactElement

Gli oggetti ReactElement si creano tramite l'uso di **espressioni JSX/TSX**

```
const elem = <espressione JSX/TSX>
```

Un'espressione JSX/TS è un frammento di **HTML (*) interpolato** con espressioni JS/TS inserite fra parentesi graffe { }. Un esempio molto semplice può essere:

```
const elem = <p>Pagina creata da {utente.nome} {utente.cognome}</p>
```

Come si vede nell'esempio:

- l'HTML è riportato direttamente nel codice, senza virgolette o altro.
- all'interno delle parentesi graffe (interpolazione) ricompare Javascript/Typescript; il codice interpolato deve risolversi con un'espressione il cui valore viene usato per completare l'HTML nel punto dove appaiono le parentesi { }

Espressioni JSX/TSX

Un'espressione JSX/TSX deve risultare in esattamente **un sottoalbero del DOM**; deve quindi esserci un elemento ben preciso a fare da radice.

```
<h1>Titolo</h1><p>Paragrafo</p>
```

non è un'espressione JSX/TSX valida, perché il risultato sarebbero due sottoalberi indipendenti, uno con radice H1 e uno con radice P. Nel caso ci serva proprio restituire un frammento del DOM fatto di due radici separate, possiamo racchiuderle con il **tag vuoto** `<>`. Otterremo così un `ReactElement` valido che la nostra **funzione-componente** possa restituire, e il tag vuoto verrà tolto prima di inserire il nostro elemento nel DOM:

```
<><h1>Titolo</h1><p>Paragrafo</p></>
```

In generale, possiamo usare il normale HTML che già conosciamo nelle espressioni JSX/TSX, con un paio di accorgimenti:

- tutti i tag devono essere chiusi, anche quelli che non prevedono un contenuto, quindi ad es: ``
- i nomi di attributi scritti in *kebab-case* (es: `data-value`) vengono convertiti in formato *camelCase* (es: `dataValue`); essi infatti diventano **proprietà** all'interno di oggetti JS/TS, e sappiamo che i nomi di proprietà non ammettono lo hyphen (-)
- l'attributo **class** viene sostituito con **className**, questo perché la parola **class** in JS/TS è una **keyword riservata**

Interpolazione

Come già detto, è possibile **interpolare** espressioni Javascript/Typescript nell'HTML di un'espressione JSX/TSX, inserendole all'interno di parentesi graffe. Questo può avvenire sostanzialmente in tre punti dell'HTML.

- come **intero contenuto** di un tag: `<ul>{espressione}</ul>`
- come **porzione di testo** all'interno di un tag: `<p>{nome} ha vinto {x+y} euro!</p>`
- come **valore di un attributo** subito: `<div className={val}>...</div>`

Il valore risultante dall'espressione può venire, in tutti e tre i casi, convertito in una stringa, e utilizzato come tale (quindi, come testo che verrà visualizzato, o come valore di un attributo).

Nei primi due casi, tuttavia, il valore può anche essere a sua volta un `ReactElement` (ossia, il risultato di un'altra espressione JSX/TX) o anche un **array** di `ReactElement`.

In quel caso l'effetto sarà di inserire l'elemento o gli elementi corrispondenti nel punto del DOM identificato dalle parentesi graffe:

```
const lista = [<li>Uno</li>,<li>Due</li>,<li>Tre</li>];  
return <ul>{lista}</ul>;
```

Il terzo caso, quello in cui l'interpolazione appare nel valore di un attributo, è quello che ci permette di associare un comportamento agli eventi della UI (quello che, senza React, facevamo tramite i listener). Per ogni evento che può avvenire su un dato elemento (ad es.: click, mousedown, keypress...) è definito un corrispondente attributo (`onClick`, `onMouseDown`, `onKeyPress`). A tale attributo è possibile assegnare come valore **il relativo listener**, proprio tramite interpolazione:

```
<button onClick={(e) => myFunction(e, ...)}>PREMI!</button>
```

Property binding ed Event binding

L'interpolazione è il meccanismo con cui il componente (che abbiamo detto essere il ViewModel nel contesto del pattern MVVM) si «lega» alla View, attraverso i due tipi di «legame» o «binding» che abbiamo descritto precedentemente:

- Il **property binding** si ha quando un componente React riporta all'interno della propria View i dati in suo possesso. Per il momento non abbiamo ancora capito come far arrivare a un componente dati «esterni» (spoiler: il componente è una funzione, la funzione potrà prendere degli input...). Abbiamo però visto come riportare all'interno della view il contenuto di una variabile, ossia tramite interpolazione. L'abbiamo fatto con le variabili locali, ma sarà esattamente lo stesso con i parametri di input della funzione.
- L'**event binding** si ha quando il componente React assegna un listener a un evento UI all'interno della sua porzione di View. Anche in questo caso ci manca un pezzo: non sappiamo come far sì che il listener modifichi qualcosa che non sia (di nuovo) una variabile locale del componente stesso. Per il momento lo **scope** dei nostri componenti contiene solo variabili locali! Se anche il listener modificasse una di esse (e potrebbe farlo, per via del meccanismo di **closure**) questa resterebbe un'operazione fine a se stessa. Per ridisegnare la view con l'ipotetico nuovo valore, infatti, React dovrebbe chiamare nuovamente la **funzione-componente**. Come risultato, la variabile verrebbe **ridichiata** (e dunque, resettata) e qualsiasi modifica avevamo apportato andrebbe perduta. spoiler: ogni componente potrà avere uno stato, memorizzato all'esterno dello scope del componente ma comunque accessibile; i listener potranno modificare lo stato e il codice che genera la view potrà leggerlo.

Gerarchia di componenti

Genitori e figli

un componente può costruire la propria **view** a partire da (una o più istanze di) altri componenti di cui sarà il **genitore**; essi saranno i suoi **figli**

```
function App() {  
  return (<>  
    <Header/>  
    <section className="main">  
      <div className="left-col">  
        <ContentNav/>  
      </div>  
      <div className="center-area">  
        <Bookshelf/>  
      </div>  
    </section>  
    <Footer/>  
  </>)  
}
```

```
export function Header(): ReactElement {...}
```

```
export function ContentNav(): ReactElement {...}
```

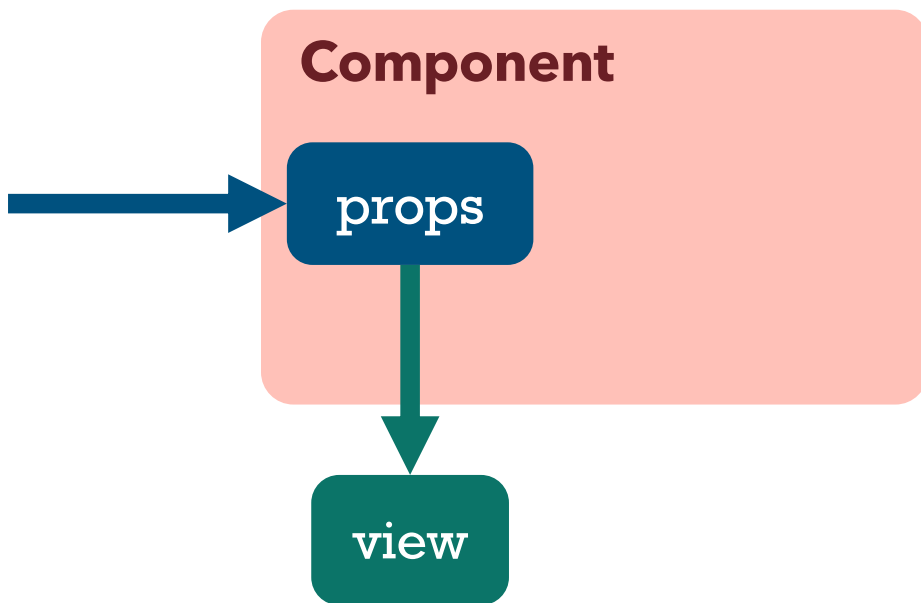
```
export function Bookshelf(): ReactElement {...}
```

```
export function Footer(): ReactElement {...}
```

Input di un componente

props

un componente può dichiarare dei parametri che il **genitore**, nell'istanziarlo, dovrà valorizzare: collettivamente essi sono detti **props** del componente.



```
interface BookProps {  
  bIS: BookInBookshelf;  
}  
  
function Book({bIS}: BookProps)  
  : ReactElement  
{...}
```

```
function Bookshelf(...): ReactElement {  
  const bookElems =  
    books.map(book =>  
      <Book bIS={book} />  
    );  
  
  return (<div className="libreria">  
    ...  
    {bookElems}  
  </div>);  
}
```

Input di un componente

props

un componente può dichiarare dei parametri che il **genitore**, nell'istanziarlo, dovrà valorizzare: collettivamente essi sono detti **props** del componente.

```
interface BookProps {  
  bIS: BookInBookshelf;  
}  
  
function Book({bIS}: BookProps)  
  : ReactElement  
{...}
```

```
function Bookshelf(...): ReactElement {  
  const bookElems =  
    books.map(book =>  
      <Book bIS={book} />  
    );  
  
  return (<div className="libreria">  
    ...  
    {bookElems}  
  </div>);  
}
```

Le «props» dei Componenti

Un componente può prendere come argomento un oggetto **props** le cui **proprietà** costituiscono i parametri di input del componente. La «forma» dell'oggetto in questione (ossia, quali proprietà esso possieda) dovrà essere dichiarata in una opportuna **interface** che andrà a costituire il **tipo** del parametro **props** in input alla funzione-componente.

Il componente riceve i **valori** delle props dal componente genitore. I valori delle props vengono assegnati nel momento in cui il componente viene **istanziato** (quindi, vengono assegnati dal genitore). Diverse istanze dello stesso componente possono quindi ricevere valori diversi, e questa differenza si rifletterà nelle rispettive **view**.

Alcuni vantaggi dell'uso delle **props** sono abbastanza evidenti:

- **Riuso**: lo stesso componente può essere utilizzato in circostanze diverse, su dati diversi
- **Configurabilità**: il componente diventa configurabile da parte del genitore, che attraverso il passaggio di props può determinare alcuni aspetti della view del figlio
- **Gestione del flusso di informazioni**: ogni componente può conoscere solo quegli aspetti del data model che influenzano la sua view; il ruolo del genitore è anche quello di stabilire come «spartire» i dati in suo possesso fra i propri figli.

C'è però un effetto dell'uso di **props** che è **centrale** in React e che supera, come utilità, quelli precedentemente menzionati:

se il valore di una proprietà dell'oggetto **props** cambia, React rigenera la **view** del componente, invocando nuovamente la funzione corrispondente con i nuovi valori

Passare «props» ad un Componente figlio

Quando si istanzia un componente all'interno di un genitore, il nome del componente diventa un tag nello pseudo-HTML utilizzato per definire la view.

I nomi delle proprietà dell'oggetto **props** (quindi, i nomi dei parametri di input) diventano **attributi** associati a quel tag. Per cui il genitore può specificare i valori delle props dei suoi figli assegnando quei valori agli attributi corrispondenti.

Figlio:

```
interface CompProps {  
    nome: string;  
}  
function Comp(props: CompProps) {...}
```

Genitore:

```
function OtherComp() {  
    return (...  
        <Comp nome="magomerlino"/>  
        ...);  
}
```

Piccola parentesi: object destructuring

Accenniamo brevemente a una caratteristica del linguaggio Javascript/Typescript che si vede spesso in azione nelle definizioni dei componenti React: l'object destructuring.

Molto semplicemente, immaginiamo di aver da qualche parte costruito un oggetto **pippo** che ha due proprietà **x** e **y**. Non è rilevante come sia stato costruito (se come object literal o con un costruttore di classe), e non è nemmeno rilevante se oltre ad **x** e **y** abbia altre proprietà.

L'object destructuring è un costrutto sintattico che permette di scrivere:

```
const {x, y} = pippo
```

dichiarando così simultaneamente due variabili **x** e **y** che contengono esattamente i valori delle due proprietà omonime di **pippo**.

Lo stesso costrutto si può usare nei parametri formali di una funzione. Se scriviamo:

```
function fun({x, y}) {...}
```

Chi **invoca** la funzione **fun** passerà un unico parametro che dovrà avere le due proprietà **x** e **y**. All'interno del codice di **fun** saranno disponibili due variabili locali **x** e **y** che assumeranno i valori delle due proprietà omonime dell'oggetto. Il risultato è equivalente al seguente codice:

```
function fun(ogg) {  
    const x = ogg.x;  
    const y = ogg.y;  
    ...  
}
```

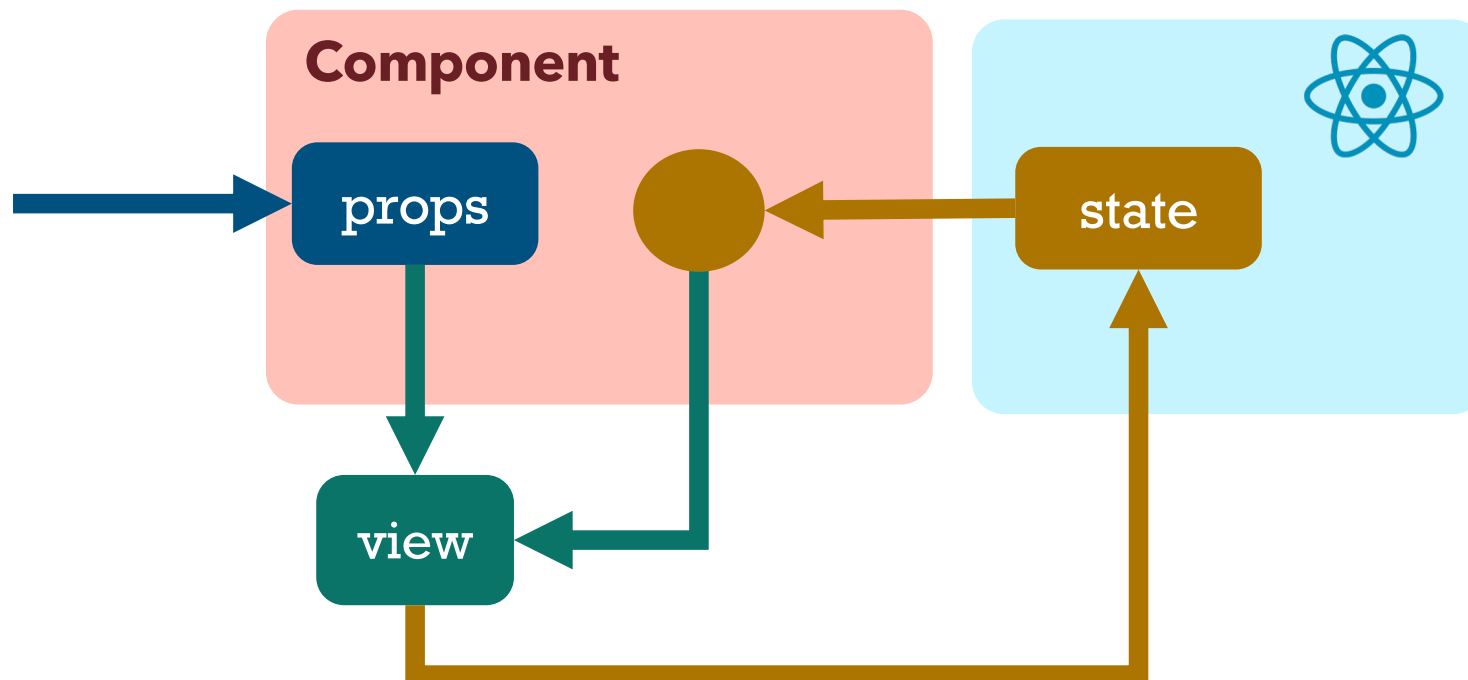
Il destructuring è un costrutto sintattico che si può applicare sia agli oggetti che agli array. Si può usare in molte forme alternative, qui ne abbiamo presentata solo una.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment

Stato di un componente

React state

un componente può registrare il proprio **stato** presso un apposito **storage** gestito da React, può **leggerne** il valore, e può dichiarare **callback/listener** che lo **modificano**



Stato di un componente

React state

un componente può registrare il proprio **stato** presso un apposito **storage** gestito da React, può **leggerne** il valore, e può dichiarare **callback/listener** che lo **modificano**

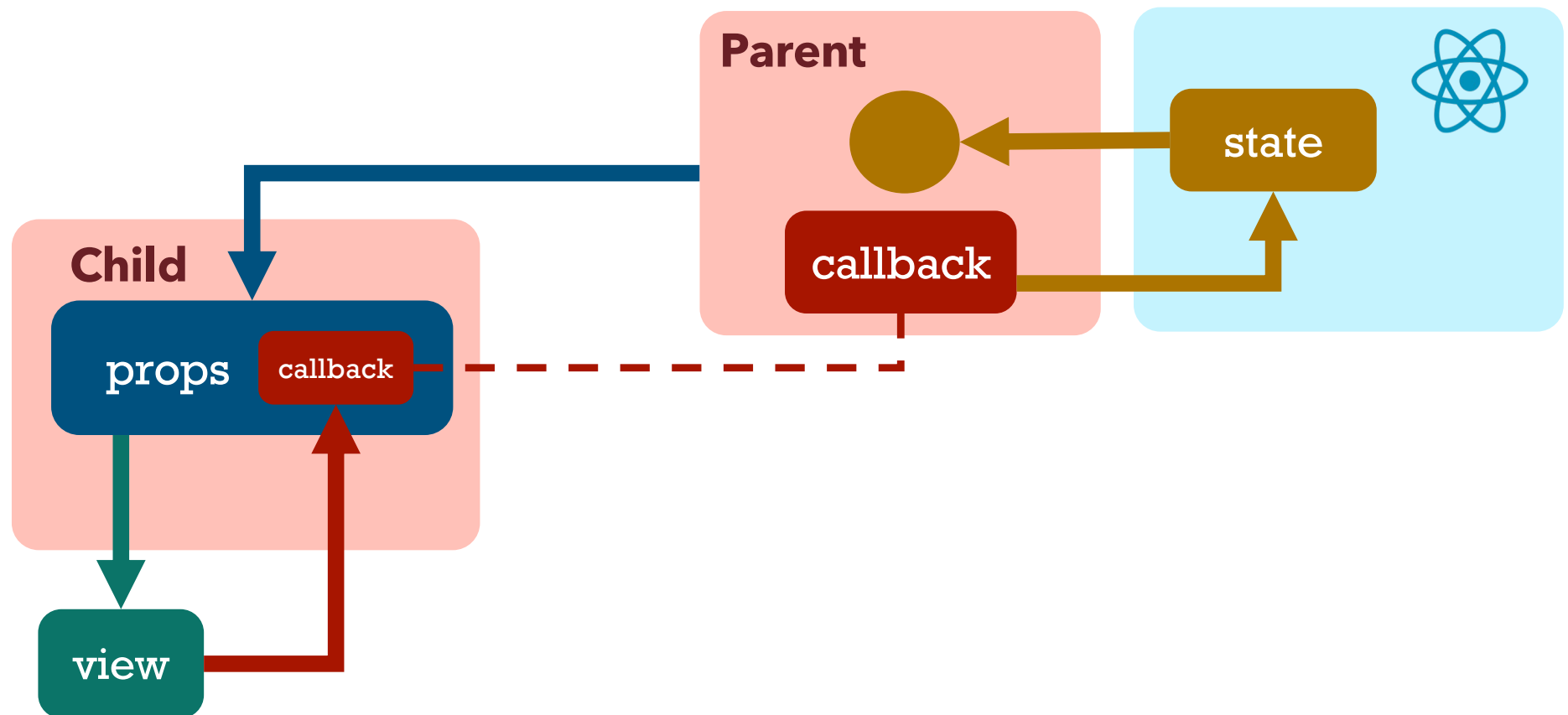
```
type ContentNavItem = "negozio" | "libreria" | "lend" | "borrow" | "amici";

export function ContentNav(): ReactElement {
  const [activeItem, setActiveItem] = useState<ContentNavItem>("libreria");
  ...
  return (<nav className="content-nav">
    <div className={ (activeItem === "negozio" ? ...) }
      onClick={() => setActiveItem("negozio")}>
      Negozio
    </div>
```

Stato di un componente

Delega al genitore

Talvolta lo **stato** di un componente si deve modificare in conseguenza di **eventi UI** che avvengono in uno dei suoi **figli**: sarà allora necessario che il figlio **deleghi** (anche parzialmente) la gestione dell'evento al genitore.



Lo «state» di un Componente

In base a quanto abbiamo visto ora, la **view** risulta **statica**: è vero che i componenti possono cambiare la propria **view** in base alle **props**, ma una volta che l'applicazione è in esecuzione non c'è modo di modificare tali valori a partire dagli eventi UI.

Quello che manca è un concetto di **stato** di un componente che possa cambiare nel tempo e influenzare la **view**. Come si concilia questo con il fatto che i componenti sono **funzioni** e per di più **funzioni pure**?

- I componenti React possono **dichiarare** di avere bisogno di una o più variabili di stato, dichiararne il tipo, e il valore **di partenza**
- il **valore corrente** delle variabili di stato di tutti i componenti è conservato (tramite l'invocazione di apposite funzioni delle librerie di React) in un deposito esterno ai componenti; per semplicità diciamo che è conservato «da React» un po' come se React fosse una sorta di ente deputato alla gestione delle faccende dei componenti.
- ad ogni esecuzione della funzione, il componente ottiene «da React» il valore corrente delle variabili di stato, che può usare nel determinare la propria **view** alla stregua degli altri input
- il componente ottiene «da React» anche una funzione da usare per **modificare** lo stato; questa funzione **non può essere invocata** nel codice del componente stesso, ma può essere invocata **nelle callback asincrone** che il componente definisce.

Questo perché l'invocazione della funzione di modifica dello stato provoca il re-rendering del componente, ossia una nuova esecuzione della funzione-componente. Se modifichiamo lo stato all'interno del codice del componente provochiamo un loop infinito di chiamate.

Cos' come per le **props**, anche per lo state di un componente vale che:

se il valore di uno **state** cambia, React rigenera la **view** del componente, invocando nuovamente la funzione corrispondente, che dunque opererà il valore di quello state aggiornato

Lo hook useState

All'interno di un componente, possiamo creare una variabile di stato usando la funzione di libreria **useState<T>** (dove T è il tipo della variabile di stato che vogliamo creare).

- **useState** prende come parametro il **valore iniziale della variabile di stato** (che dunque dovrà corrispondere al tipo T); questo valore viene utilizzato solo **la prima volta** che la funzione-componente viene eseguita, ossia quando la variabile di stato non esiste ancora e deve essere inizializzata. Ad esempio:

```
const infoDiStato = useState<number>(0);
```

- **useState** restituisce una coppia di valori (tecnicamente, un array con due elementi):
 - il **valore corrente** della variabile di stato, che la prima volta corrisponderà al valore iniziale, ma successivamente potrà avere altri valori, sempre del tipo dichiarato T.
 - una **funzione** che potrà essere usata (con le restrizioni precedentemente descritte) per modificare il valore dello stato

```
const [myState, setMyState] = infoDiStato;
```

(per inciso, questa riga di codice è un esempio di **array destructuring**: alle due variabili a sinistra vengono assegnati i primi due valori dell'array a destra; eventuali valori aggiuntivi vengono ignorati)

La funzione restituita («**setter**») è una funzione che riceve un valore di tipo T e lo fa diventare il **prossimo valore** della variabile di stato. Può anche ricevere una funzione «di aggiornamento», che determina il nuovo valore sulla base del precedente. La funzione di aggiornamento prende come argomento un T e restituisce un altro T.

useState è un «**React hook**» ossia un **gancio** fra il nostro codice e le funzionalità attinenti al **ciclo di vita** del componente così come viene interamente gestito «da React».

Delega di un evento al componente genitore

Talvolta lo **stato** di un componente si deve aggiornare in seguito ad **eventi** che avvengono nei suoi **figli**.

Parliamo in questo caso di Prendiamo l'esempio di una lista in cui può essere selezionato uno e un solo elemento. In questo esempio, ogni elemento della lista è un componente a sé e si accorge di quando viene clickato, per cui può «auto-selezionarsi». Però per poter deselezionare gli altri elementi è necessaria una visione di insieme, che è in possesso solo del componente che contiene l'intera lista.

Sarà il componente-lista dunque ad avere una variabile di stato che registra **quale** degli elementi è selezionato. Ciascun componente figlio dovrà avvisare il genitore quando viene selezionato, in modo che il genitore possa gestire la visione d'insieme.

Si parla dunque di **delega** al genitore: al verificarsi di un evento nel componente figlio, sarà possibile chiamare una **callback** che non appartiene al figlio, bensì al genitore.

Sarà anche possibile chiamare due **callback**, una del figlio e una del genitore, o chiamare una **callback** del figlio che a sua volta invoca la **callback** del genitore. In altre parole, l'evento può essere delegato in toto o in parte, permettendo al figlio di scegliere se, oltre alla delega, ha dei cambiamenti di stato propri che vuole mettere in atto.

Per rendere possibile la **delega**, il figlio dichiarerà di ricevere fra le sue **props** anche una funzione definita dal genitore: la **callback** che dovrà essere chiamata al verificarsi dell'evento. Per convenzione, le **props** demandate alla gestione degli eventi, che dunque hanno valori di tipo funzione, hanno nomi che iniziano per **on** seguito dal nome dell'evento stesso.

Livelli di astrazione negli eventi

Quando un componente **delega** un evento che avviene al suo interno, si dice anche che **espone** l'evento in questione al suo genitore. Il **nome** dell'evento esposto deve, per chiarezza del codice, essere usato nel **nome della prop** all'interno della quale il genitore fornirà la callback. Quindi, se decidiamo che l'evento esposto si chiama «action», la **prop** si chiamerà **onAction**.

Il figlio può esporre direttamente l'evento UI da esso stesso ricevuto. Nell'esempio precedente, il componente figlio, cioè la singola voce della lista, può chiamare la callback del genitore ad ogni *click* che riceve. In tal caso l'evento esposto è a tutti gli effetti il *click*, e la prop che riceve la callback si chiamerà **onClick**.

Anche l'attributo dell'elemento HTML a cui passiamo la callback del figlio si chiama `onClick`: in effetti, quelli che appaiono come elementi HTML in JSX, sono in realtà a loro volta componenti interni di React, e gli attributi sono le loro **props**!

Può però anche succedere che il figlio **elabori** gli eventi UI da lui ricevuti, e a partire da essi determini un evento ad un livello di astrazione più alto, che si verifica solo a certe condizioni o con certe combinazioni degli eventi UI di livello inferiore.

Sempre nell'esempio del componente-lista, supponiamo che il componente-lista sia a sua volta figlio di un genitore che è interessato a sapere quando la selezione si modifica. Non è utile che il componente-lista inoltri al genitore tutti i click sui suoi sotto-componenti, con tanto di posizione del puntatore del mouse e dettagli sui tasti che erano premuti durante il click. Piuttosto, il componente-lista chiamerà la callback del genitore solo quando effettivamente avviene la *selezione* di una nuova voce. Se l'utente clicca tre volte sulla stessa voce, solo il primo click opera effettivamente una nuova selezione.

In questo caso possiamo dire che il componente-lista **espone un evento di livello più alto**, che potremo ad esempio chiamare «selection». La **prop** corrispondente si chiamerà dunque **onSelection**. In questo modo i programmatori possono definire i cosiddetti «eventi custom», ossia eventi complessi, specifici dei loro componenti e/o della loro applicazione, che dunque non rientrano fra quelli già previsti dall'ambiente browser o da React.

Cenni ai React Hooks

useState non è l'unico **hook** definito da React. In generale sono dette **hook** le funzioni offerte dalla libreria di React per **agganciarsi** (in una varietà di modi e per una varietà di scopi) al processo con cui React gestisce i componenti e influire parzialmente su di esso.

useState ad esempio restituisce la funzione **setter** che, quando invocata, oltre a modificare il valore dello stato del componente, ne forza il refresh.

Altri **hook** sono: **useReducer**, **useContext**, **useRef**, **useMemo**, **useCallback**, **useEffect**...

<https://react.dev/reference/react/hooks>

Come si può notare gli **hook** sono riconoscibili per il fatto che i loro nomi iniziano sempre per «**use**». È anche possibile per il programmatore definire **propri «custom» hook** a partire da quelli esistenti ma non entreremo nel dettaglio di questo.

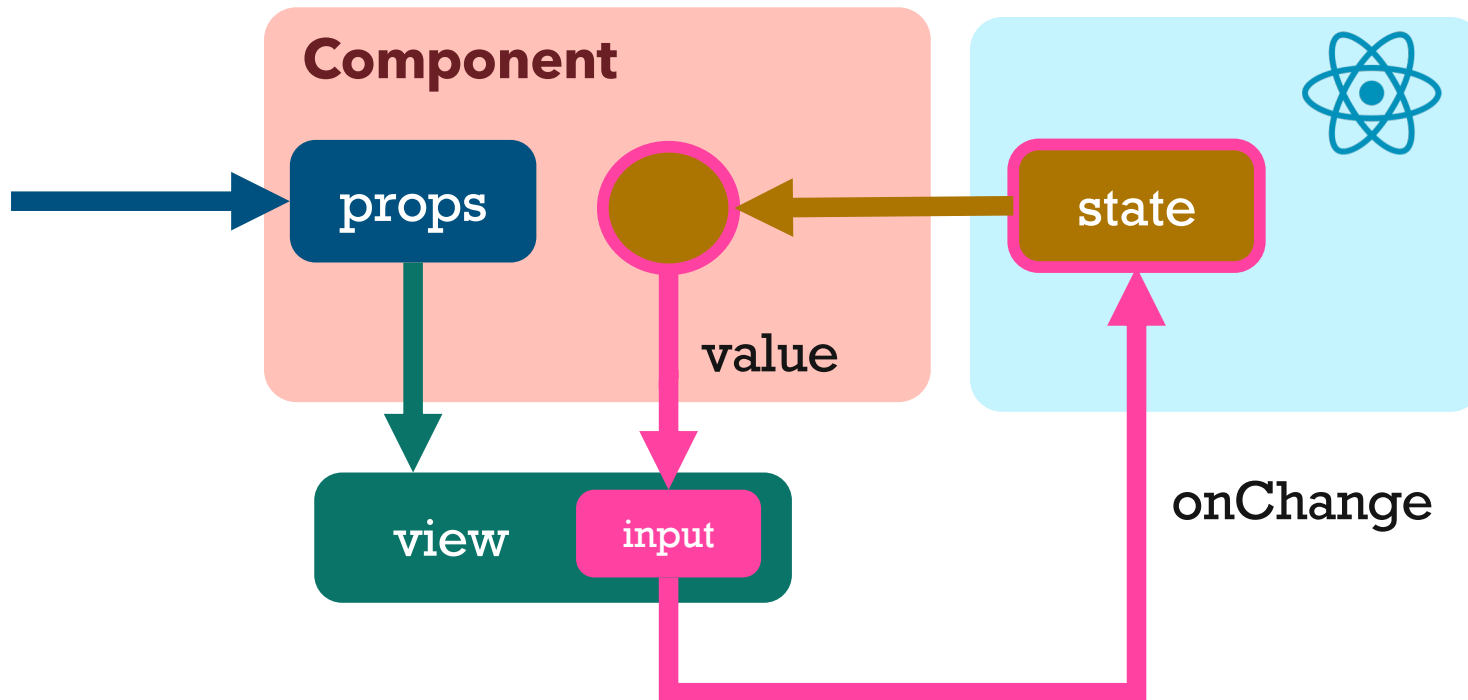
Gli **hook** non possono essere usati dovunque nel nostro codice, soltanto:

- nello scope principale di una **funzione-componente**
- nello scope principale di un **custom hook**

form con React

come gestire gli elementi di input del DOM

Quando la **view** contiene un elemento di **input** dobbiamo garantire una sincronizzazione **bidirezionale** fra il **valore** dell'elemento e un corrispondente **state** del componente



Gestire i form in React: binding bidirezionale

Quando abbiamo un **form** in una single page application non vogliamo appoggiarci al meccanismo automatico del browser per gestire i form e i relativi submit, perché esso provoca un **reload** della pagina con relative conseguenze.

Pertanto, gestiamo il form utilizzando gli strumenti di React.

In React possiamo definire **form**, o **form-like**, qualunque componente contenga degli elementi di **input** (quindi, in HTML, elementi INPUT o SELECT o TEXTAREA).

Anche pulsanti di conferma o annullamento (o reset) del «form» non dovranno essere necessariamente degli INPUT di tipo "submit" ma potranno essere realizzati con qualsiasi elemento cliccabile (quindi, con qualsiasi elemento HTML...)

I passi necessari per gestire un componente **form-like** sono i seguenti:

1. Per ciascun elemento di input nella **view** del componente dichiariamo uno **state** atto a contenere il valore di quell'elemento:
 - un `<input type="text">` avrà come valore la **string** corrispondente al testo inserito
 - un `<select>` avrà come valore l'attributo **value** della `<option>` selezionata (quindi, di nuovo una **string**)
 - un `<input type="checkbox">` avrà come valore un **boolean** (selezionata o meno)
2. Assegniamo a ciascun elemento di input l'attributo **value**, facendolo corrispondere al valore dello **state** corrispondente: `<input type="text" value={stateVar}>`
3. Riceviamo l'evento **onChange** su ciascun input, e lo usiamo per modificare il valore dello **state** corrispondente:

```
<input type="text" value={stateVar} onChange={(e) => setStateVar(e.target.value)}>
```

Questo è sia un **property binding** (la variabile **stateVar** viene passata per interpolazione all'elemento di input) sia un **event binding** (la variabile **stateVar** viene settata in conseguenza di un evento; si parla in questo caso di **doppio binding** o **binding bidirezionale**).

Gestire i form in React: attuare gli obiettivi del form

Possiamo grosso modo distinguere tre tipologie di componenti **form-like** in base al modo in cui si attua l'obiettivo del form:

- Tipologia «pannello di controllo»: il **form-like** è presente nella view; contiene diversi elementi di controllo e il modo in cui sono impostati si riflette sulle view degli altri componenti sulla pagina

In questo caso la delega al genitore avverrà quasi ad ogni cambiamento di stato interno al componente form-like, perché qualsiasi variazione dello stato interno si rifletterà in una variazione di uno stato del componente-genitore, che a sua volta si rifletterà nelle props passate agli altri figli in modo che essi possano modificare la view di conseguenza.

Esempio: un insieme di filtri per categoria o tipologia che, in base alla selezione di opzioni dell'utente visualizza certe cose e non altre.

- Tipologia «modulo»: il **form-like** è presente nella view; contiene un modulo la cui «azione» si compie se l'utente preme il pulsante di conferma.

In questo caso la delega al genitore avviene solo quando l'utente preme effettivamente il pulsante di conferma. Tutti gli altri eventi interni modificano solo lo stato interno del componente form-like. Anche un eventuale pulsante di «reset» dei contenuti del form può essere gestito internamente dal componente form-like.

Esempio: un form di iscrizione a una newsletter, che è sempre presente in una colonna laterale.

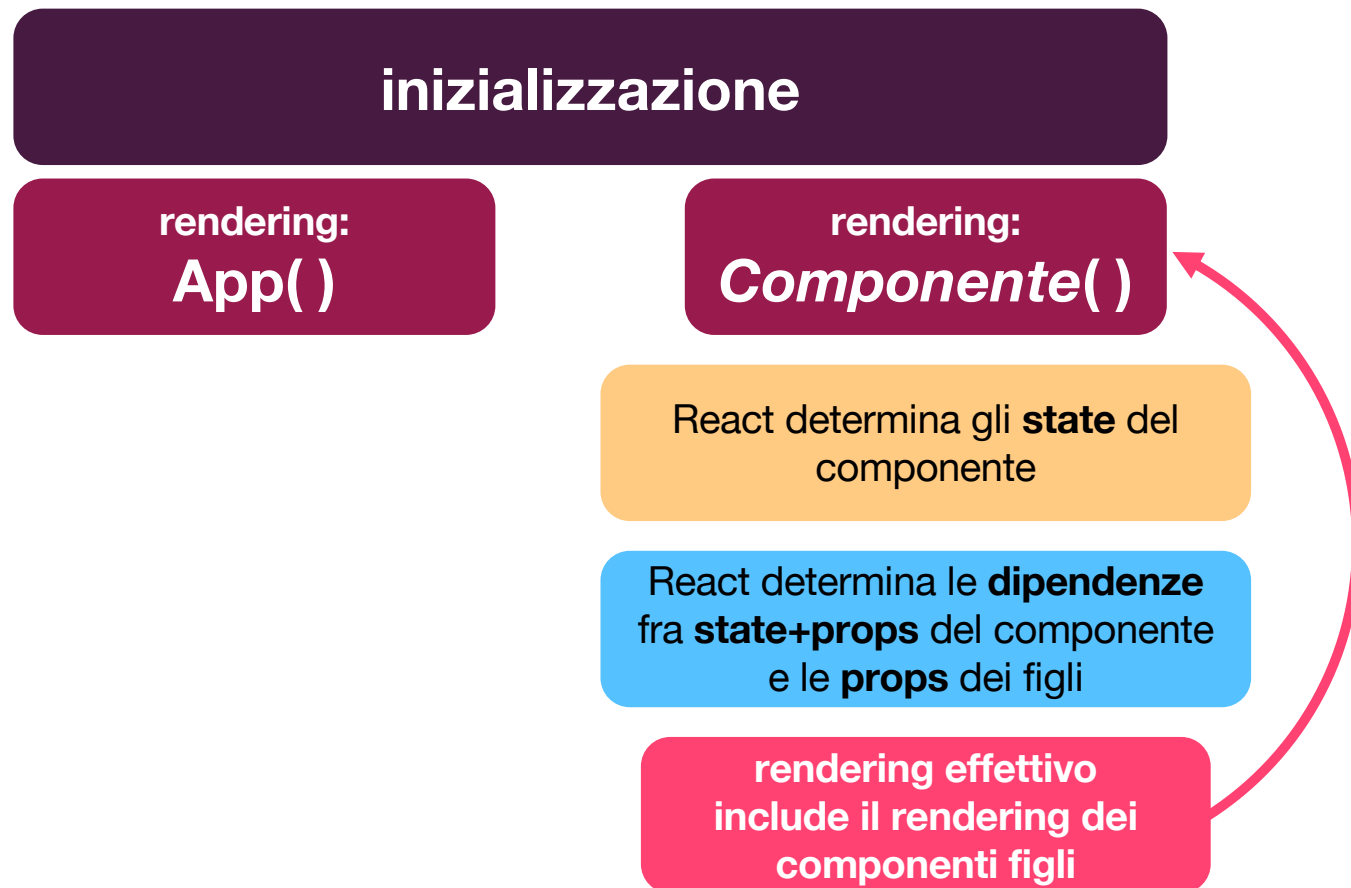
- Tipologia «modal dialog»: il **form-like** non è presente nella view, appare per richiedere dati all'utente e lì rimane finché l'utente non conferma o annulla

Il fatto che il componente form-like sia visibile o meno in questo caso è responsabilità del genitore. Infatti un componente invisibile non ha modo di cambiare il proprio stato, solo il genitore può farlo. La delega al genitore dovrà avvenire in corrispondenza degli eventi che confermano l'azione, ma anche degli eventi che la annullano, cosicché il genitore possa far scomparire la dialog box.

Esempio: una dialog che permette di inserire i dati per il pagamento di un ordine

il ciclo di rendering di React

1.inizializzazione



il ciclo di rendering di React

2.cambiamenti di stato

invocazione di **setVar** per qualche variabile di stato **var**

React determina a quale **componente** appartiene lo stato **var**

re-rendering:
Componente()

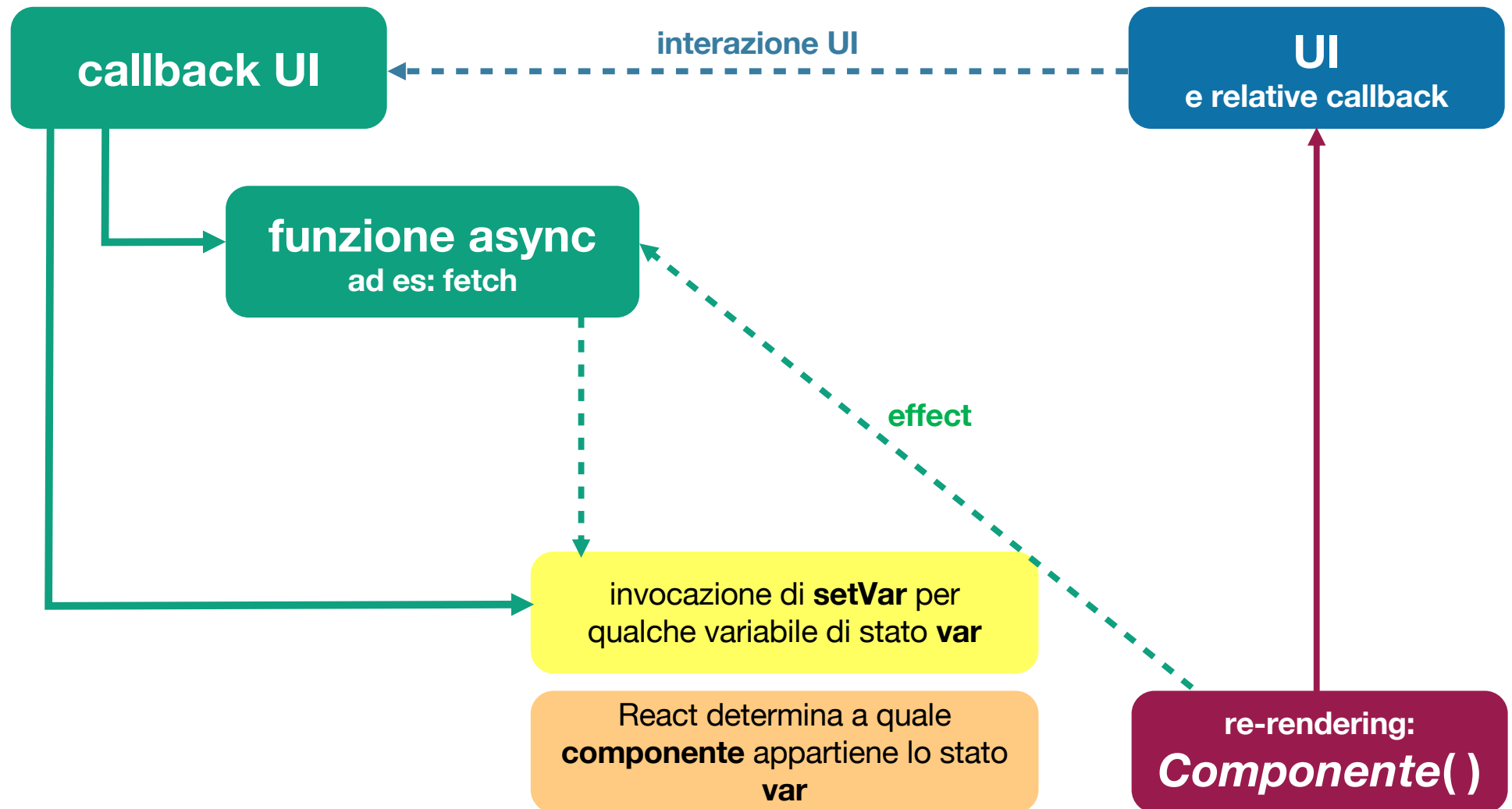
React determina le conseguenze del cambiamento di **var** su altre variabili/espressioni della funzione di rendering

React determina quali **props** dei componenti figli risultano modificate

re-rendering effettivo
solo di ciò che è cambiato:
elementi montati/smontati
elementi i cui attributi sono cambiati
figli le cui props sono cambiate

il ciclo di rendering di React

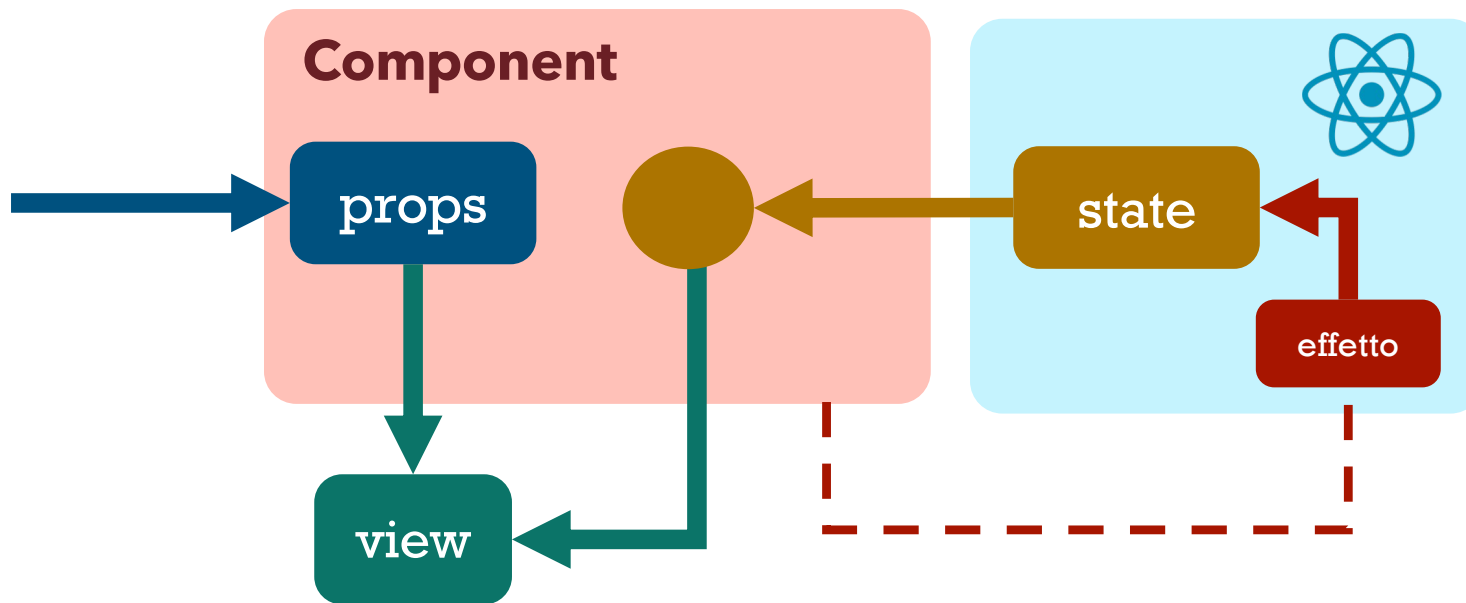
cosa provoca i cambiamenti di stato?



useEffect

attuare effetti «collaterali» del rendering in modo asincrono

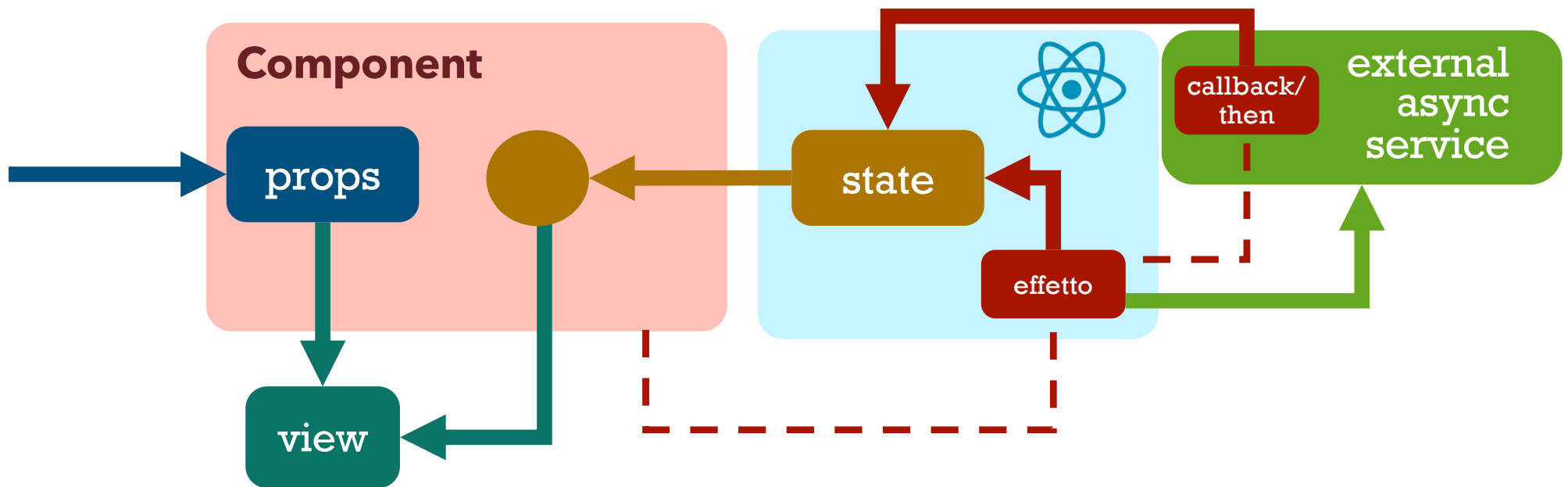
Un **effetto** in React è un blocco di codice che deve essere eseguito in conseguenza di certi **cambiamenti** di **props** o **state**, ma che non può essere eseguito direttamente nella funzione-componente, e viene pertanto eseguito come **funzione asincrona** in seguito all'invocazione della funzione di rendering.



useEffect

attuare effetti «collaterali» del rendering in modo asincrono

Un **effetto** in React è un blocco di codice che deve essere eseguito in conseguenza di certi **cambiamenti** di **props** o **state**, ma che non può essere eseguito direttamente nella funzione-componente, e viene pertanto eseguito come **funzione asincrona** in seguito all'invocazione della funzione di rendering.



useEffect

Un **effetto** in React è qualcosa che deve accadere come conseguenza del re-rendering del **componente** ma che non può essere **eseguito direttamente** nella **funzione di rendering** in quanto **influisce sul mondo esterno** al componente (e dunque viola la purezza della funzione) o perché può causare un **rerendering** (ad esempio perché cambia lo stato del componente, ma non è l'unico caso) o comunque **interferire** col rendering stesso.

Ecco alcuni esempi:

- **fetch** di un dato necessario alla visualizzazione del componente
- **sottoscrizione** di un servizio/funzionalità esterna al componente, con relativa associazione di funzioni di callback che da esso possono essere invocate
- **logging** di dati ad es. su local DB
- **gestione** di effetti di animazione

Se abbiamo necessità di un effetto di questo tipo (e ce l'avremo praticamente tutte le volte che dobbiamo effettuare una **fetch** dei dati necessari al componente), dobbiamo usare un **React hook** denominato **useEffect**.

useEffect(funzioneEffetto, dipendenzeEffetto?)

- **funzioneEffetto** è la funzione che attua l'effetto.
- **dipendenzeEffetto** è un **array opzionale** che elenca le **variabili** dello scope locale da cui l'effetto dipende (ossia il cui cambiamento richiede la ri-esecuzione dell'effetto).

Se l'array è **vuoto**, l'effetto verrà eseguito solo al **primo** rendering.

Se l'array **non viene specificato**, l'effetto verrà eseguito ad **ogni** rendering.

La **funzioneEffetto** viene inserita nella coda di eventi asincroni, dunque viene eseguita quando il rendering è ormai terminato.

useEffect in combinazione con setState

Abbiamo detto a **funzioneEffetto** viene inserita nella coda di eventi asincroni, dunque viene eseguita quando il rendering è ormai terminato.

Essendo eseguita in modo asincrono è possibile al suo interno invocare **setState**. Bisognerà però fare attenzione a far sì che non si inneschi un loop infinito fra cambiamenti di stato provocati dall'effetto e dipendenze dell'effetto stesso.

- Nel caso più semplice, **setState** modifica una variabile di stato che **non** influisce sulle **dipendenze dell'effetto**, per cui la modifica dello stato **non** comporta una riesecuzione dell'effetto stesso
- Un caso un po' più complesso, ma comunque gestibile, è quello in cui effettivamente la modifica dello stato influisce sulle dipendenze dell'effetto, che dunque viene rieseguito. In questo caso sarà necessario che la modifica dello stato non avvenga sistematicamente ad ogni esecuzione dell'effetto, ma sia sottoposta a una **condizione** che prima o poi diventa falsa. In questo caso si avrà un loop, ma non sarà un loop infinito, paragonabile a un ciclo **while**.

In ogni caso, la combinazione di **useEffect** e **setState** è spesso inevitabile ma va gestita con cautela. Il caso più comune è quello in cui:

- al primo rendering, e talvolta anche in seguito alla modifica di una **prop**, il componente deve effettuare una **fetch** dal server di un certo **dato**
- la **fetch** viene eseguita in un **effetto F**; quando i dati arriveranno (**then**) verranno comunicati al componente tramite una **setState(dato)**
- in seguito all'arrivo del **dato** e del relativo **setState(dato)** avviene il **rerendering** del componente
- è dunque opportuno specificare le dipendenze dell'**effetto F** in modo che il cambiamento di **dato** non provochi la sua riesecuzione: fra le dipendenze di **F** non ci deve ovviamente essere **dato**, ma nemmeno nessun'altra **variabile locale** che viene calcolata a partire da **dato** e che dunque potrebbe risultare modificata, provocando la riesecuzione di **F**

useEffect e funzione di cleanup

Alcuni effetti hanno bisogno di «fare le pulizie» prima di poter essere rieseguiti.

Si pensi ad esempio alla sottoscrizione di un servizio esterno o alla connessione ad un server di terze parti: prima di operare una nuova iscrizione o connessione, sarà opportuno terminare la precedente prima di essere bombardati dalle notifiche o causare qualche memory leak.

Nel ragionare su questo, va considerato che un effetto potrebbe essere rieseguito per «colpa» di un rerendering **prima** che il risultato dell'esecuzione precedente si sia compiuto. In questo caso ci può essere bisogno di **annullare** o **invalidare** l'esecuzione precedente.

Prendiamo un effetto che richiede una **fetch** di un dato che dipende da una **prop** del componente. Immaginiamo il caso in cui la **prop** viene aggiornata, e il dato viene nuovamente richiesto al server, **prima** che la **fetch** precedente sia completata. A questo punto ci sono in corso **due fetch** per due versioni diverse del dato, e non possiamo sapere in che ordine arriveranno. È teoricamente possibile che la prima **fetch** (che non vogliamo più) arrivi **dopo** la seconda, e che il dato **obsoleto** sovrascriva quello corretto.

C'è infine il caso in cui un componente viene **smontato** prima che un suo effetto si sia compiuto. In quel caso l'effetto potrebbe cercare di modificare uno **state** che non esiste più: React rimuove gli **state** dei componenti non più validi. Anche in questo caso bisognerà annullare o invalidare l'esecuzione ancora in corso dell'effetto.

La risposta a questi problemi è la **funzione di cleanup**. Una **funzioneEffetto** deve restituire un'altra funzione (senza argomenti né valori di ritorno) che attua l'annullamento, invalidamento o terminazione dell'effetto stesso.

React invoca la **funzione di cleanup**:

- subito prima di **reinvocare** il medesimo effetto
- subito prima di **smontare** il componente

È un aspetto talmente importante che, per verificare non venga trascurato, quando siamo in fase di sviluppo React invoca tutti gli effetti **due volte di fila**: in questo modo eventuali conseguenze delle mancate pulizie vengono più facilmente (anche se non sempre) rilevati.